

The Complete NumPy Roadmap

Step-by-Step Journey from Beginner → Advanced

Created by Goutam Kuiri | [LinkedIn](#)

1. What is NumPy?

2. Creating Arrays

NumPy provides several functions to create arrays with different patterns, values, and shapes.

Function	Description	Example Output
<code>np.array()</code>	Create array from list	<code>[1, 2, 3]</code>
<code>np.zeros()</code>	Array of zeros	<code>[[0. 0. 0.], [0. 0. 0.]]</code>
<code>np.ones()</code>	Array of ones	<code>[[1. 1. 1.], [1. 1. 1.], [1. 1. 1.]]</code>
<code>np.full()</code>	Array filled with a specific value	<code>[[7 7], [7 7]]</code>
<code>np.arange()</code>	Evenly spaced values within range	<code>[0 2 4 6 8]</code>
<code>np.linspace()</code>	Evenly spaced values between start and end	<code>[0. 0.25 0.5 0.75 1.]</code>
<code>np.eye()</code>	Identity matrix	<code>[[1. 0. 0.], [0. 1. 0.], [0. 0. 1.]]</code>

✔ **Daily Use:** Quickly initialize test data, placeholders, or structured matrices.

Example:

```
import numpy as np

np.array([1, 2, 3])
np.zeros((2, 3))
np.ones((3, 3))
np.full((2, 2), 7)
np.arange(0, 10, 2)    # → [0 2 4 6 8]
np.linspace(0, 1, 5)   # → [0. 0.25 0.5 0.75 1. ]
np.eye(3)              # → 3x3 Identity Matrix
```

3. Array Attributes – Explore Array Properties

NumPy provides useful **attributes** to quickly explore an array's **structure, size, type, and memory usage**.

Attribute	Description	Example Output
<code>arr.shape</code>	Dimensions of the array (rows, columns)	<code>(2, 3)</code>
<code>arr.size</code>	Total number of elements	<code>6</code>
<code>arr.dtype</code>	Data type of elements	<code>dtype('int64')</code>
<code>arr.ndim</code>	Number of dimensions	<code>2</code>
<code>arr.itemsize</code>	Size in bytes of each element	<code>8</code>

✔ **Daily Use:** Quickly check array dimensions, memory usage, and element data type.

Example:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

arr.shape    # → (2, 3)
arr.size     # → 6
arr.dtype    # → dtype('int64')
arr.ndim     # → 2
```

```
arr.itemsize # → 8
type('int64')
arr.ndim     # → 2
arr.itemsize # → 8
```

4. Indexing & Slicing

Definition:

Indexing & slicing are methods in NumPy to access and extract parts of an array using position-based logic.

Syntax:

```
arr[start:stop:step]           # 1D slicing
arr[row, col]                  # 2D indexing
arr[start_row:end_row, start_col:end_col] # 2D slicing
arr[condition]                 # Boolean masking
arr[[row_indices], [col_indices]] # Fancy indexing
```

Concepts:

- **1D, 2D, 3D Slicing:** Extract a portion of an array using slicing logic.
- **Fancy Indexing:** Access elements at specific positions using lists or arrays of indices.
- **Boolean Masking:** Select elements based on conditions (e.g., all elements > 3).

Daily Use: Extract data from model results or focus on specific regions of data arrays.

Examples:

```
arr[0, 1]      → 2           # Element at 1st row, 2nd column
arr[:, 1]      → [2, 5]      # All rows, 2nd column
arr[1:, :2]    → [[4, 5]]    # From 2nd row, first 2 columns
arr[arr > 3]   → [4, 5, 6]   # Boolean condition
arr[[0, 1], [1, 2]] → [2, 6] # Fancy indexing
```

Comparison Table: Forward vs Backward Indexing

Concept	Direction	Starts From	Example Index	Accessed Element
Forward Indexing	Left → Right	0	arr[2]	3rd element
Backward Indexing	Right → Left	-1	arr[-2]	2nd last element

5. Reshaping & Flattening

NumPy provides several **reshaping and flattening functions** to modify array structure, dimensions, and layout without changing data values.

Function	Description	Example
reshape()	Change array shape without changing data	arr.reshape(3, 2)
flatten()	Returns a copy of array as 1D	arr.flatten()
ravel()	Returns a view as 1D (memory efficient)	arr.ravel()
.T	Transpose rows ↔ columns	arr.T
np.newaxis	Add a new dimension (e.g., 1D → 2D)	arr[:, np.newaxis]
squeeze()	Remove size-1 dimensions (e.g., (3,1) → (3,))	np.squeeze(arr)
expand_dims()	Add a new axis at given position	np.expand_dims(arr, axis=0)

✓ **Daily Use:** Reshaping ML/DL inputs, handling image data, preparing sequences, and ensuring broadcasting compatibility.

Example:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```

arr.reshape(3, 2)      # Change shape
arr.flatten()          # 1D copy
arr.ravel()            # 1D view
arr.T                  # Transpose
arr[:, np.newaxis]     # Add new axis
np.squeeze(arr)        # Remove single-dim entries
np.expand_dims(arr, 0) # Add new axis at position 0

```

5.5. Stacking & Splitting Arrays

Concept:

Combine ya break arrays along specific axes.

Daily Use:

- Merging datasets for ML
- Creating batches for training
- Image channel combination
- Feature-label split in supervised learning

(i) Stacking Arrays

Stack karne ka matlab hota hai arrays ko chipka dena ek naye axis ke sath ya row/column ke roop mein.

Function	Purpose
<code>np.stack()</code>	Naye axis pe stack karta hai
<code>np.hstack()</code>	Horizontally (columns) stack karta hai
<code>np.vstack()</code>	Vertically (rows) stack karta hai
<code>np.dstack()</code>	Depth axis (3D stack) karta hai
<code>np.column_stack()</code>	Column-wise join (reshape-aware)
<code>np.row_stack()</code>	Row-wise join (reshape-aware)

Syntax:

```

a = np.array([1, 2])
b = np.array([3, 4])

np.stack((a, b))      # [[1, 2], [3, 4]]
np.vstack((a, b))     # [[1, 2], [3, 4]]
np.hstack((a, b))     # [1, 2, 3, 4]
np.dstack((a, b))     # [[[1, 3], [2, 4]]]
np.column_stack((a, b)) # [[1, 3], [2, 4]]
np.row_stack((a, b))  # [[1, 2], [3, 4]]

```

Hinglish Explanation:

Function	Description (English)	Explanation (Hinglish)
<code>np.stack()</code>	Stack along a new axis	Naya axis banake arrays ko jodta hai
<code>np.vstack()</code>	Stack vertically (row-wise)	Rows ke form me upar niche jodta hai
<code>np.hstack()</code>	Stack horizontally (column-wise)	Side-by-side jodta hai
<code>np.dstack()</code>	Stack depth-wise (along 3rd dimension)	3D axis (depth) me combine karta hai
<code>np.column_stack()</code>	Stack 1D arrays as columns in 2D array	1D arrays ko column-wise 2D array me banata hai
<code>np.row_stack()</code>	Stack 1D arrays as rows in 2D array	1D arrays ko row-wise 2D array me banata hai

(ii) Splitting Arrays

Splitting ka matlab hai ek array ko multiple parts me todna along axis.

Function	Purpose
<code>np.split()</code>	Specific index ke base pe split karta hai
<code>np.hsplit()</code>	Column-wise split karta hai (horizontal cut)
<code>np.vsplit()</code>	Row-wise split karta hai (vertical cut)
<code>np.dsplit()</code>	3D arrays ke depth pe split karta hai

Syntax:

```
arr = np.array([[1, 2, 3, 4],
               [5, 6, 7, 8]])

np.split(arr, 2, axis=1) # Output: [ [[1, 2], [5, 6]], [[3, 4], [7, 8]] ]
np.hsplit(arr, 2)        # 2 column-wise splits → [ [[1, 2], [5, 6]], [[3, 4], [7, 8]] ]
np.vsplit(arr, 2)        # 2 row-wise splits    → [ [[1, 2, 3, 4]], [[5, 6, 7, 8]] ]

# For 3D array example:
arr3d = np.array([[[1, 2, 3], [4, 5, 6]]])
np.dsplit(arr3d, 3)
# Output:
# [
#   [[[1], [4]]],
#   [[[2], [5]]],
#   [[[3], [6]]]
# ]
```

STAGE 2: Data Manipulation (Level: Intermediate)

Goal: Data transformation and analysis

6. Mathematical & Statistical Operations

NumPy provides powerful mathematical functions to perform element-wise operations and aggregate/statistical analysis on arrays.

Element-wise Operations

Apply directly on each element of the array.

Operation	Description	Syntax Example
Addition	Add a value to each element	<code>np.add(arr, 1)</code> or <code>arr + 1</code>
Subtraction	Subtract a value	<code>np.subtract(arr, 2)</code> or <code>arr - 2</code>
Multiplication	Multiply each element	<code>np.multiply(arr, 3)</code> or <code>arr * 3</code>
Division	Divide each element	<code>np.divide(arr, 2)</code> or <code>arr / 2</code>
Power	Raise each element	<code>np.power(arr, 2)</code> or <code>arr ** 2</code>

Daily Use: Useful for data normalization, feature scaling, transformation, etc.

Aggregation / Statistical Operations

Summarize data across the entire array or specific axis.

Function	Description	Syntax Example
<code>np.sum()</code>	Total sum of all elements	<code>np.sum(arr)</code>
<code>np.mean()</code>	Mean (average)	<code>np.mean(arr)</code>

<code>np.std()</code>	Standard deviation	<code>np.std(arr)</code>
<code>np.var()</code>	Variance	<code>np.var(arr)</code>
<code>np.min()</code>	Minimum value	<code>np.min(arr)</code>
<code>np.max()</code>	Maximum value	<code>np.max(arr)</code>
<code>np.argmin()</code>	Index of minimum value	<code>np.argmin(arr)</code>
<code>np.argmax()</code>	Index of maximum value	<code>np.argmax(arr)</code>

Daily Use: Useful for creating summary statistics, performance monitoring, and data distribution analysis.

Types of Operations:

1. Element-wise Operations

Operate on each element individually.

Operation	Syntax Example	Description
+ Addition	<code>np.add(arr, 5)</code> or <code>arr + 5</code>	Add 5 to each element
− Subtraction	<code>np.subtract(arr, 2)</code> or <code>arr - 2</code>	Subtract 2 from each element
× Multiplication	<code>np.multiply(arr, 3)</code> or <code>arr * 3</code>	Multiply each element by 3
÷ Division	<code>np.divide(arr, 2)</code> or <code>arr / 2</code>	Divide each element by 2
Power	<code>np.power(arr, 2)</code> or <code>arr ** 2</code>	Square each element

7. Conditional Filtering & Boolean Logic

NumPy allows you to apply **conditions** on arrays and get **boolean masks** or **filtered results**. It is widely used for **data cleaning**, **subsetting**, and **conditional transformations**.

Boolean Comparison Operators

Return `True` or `False` for each element based on a condition.

Operator	Description	Syntax Example
<code>arr == x</code>	Equal to	<code>arr == 5</code>
<code>arr != x</code>	Not equal to	<code>arr != 5</code>
<code>arr > x</code>	Greater than	<code>arr > 5</code>
<code>arr < x</code>	Less than	<code>arr < 5</code>
<code>arr >= x</code>	Greater than or equal to	<code>arr >= 5</code>
<code>arr <= x</code>	Less than or equal to	<code>arr <= 5</code>

✓ **Daily Use:** Filtering datasets, finding specific values, conditional data selection.

Logical Operations

Combine multiple conditions.

Function	Description	Syntax Example
<code>np.logical_and()</code>	Check if both conditions are true	<code>np.logical_and(arr > 3, arr < 10)</code>
<code>np.logical_or()</code>	Check if at least one condition is true	<code>np.logical_or(arr < 3, arr > 8)</code>
<code>np.logical_not()</code>	Negate condition	<code>np.logical_not(arr == 5)</code>
<code>np.logical_xor()</code>	Check if exactly one condition is true (exclusive OR)	<code>np.logical_xor(arr > 3, arr < 10)</code>

✔ **Daily Use:** Multi-condition filtering, advanced queries.

Conditional Filtering

Extract only elements that meet a condition.

Function	Description	Syntax Example
Boolean Indexing	Directly use condition inside brackets	<code>arr[arr > 5]</code>
<code>np.where()</code>	Return indices or elements matching condition	<code>np.where(arr > 5)</code>
<code>np.isin()</code>	Check membership of elements	<code>np.isin(arr, [1, 3, 5])</code>

✔ **Daily Use:** Data cleaning, subsetting data, handling missing or outlier values.

8. Sorting & Searching in NumPy

NumPy provides powerful **sorting** and **searching** functions for efficient data organization and retrieval. It also supports **set operations** for comparing and combining arrays.

Sorting Functions

Arrange array elements in ascending or descending order.

Function	Description	Example
<code>np.sort()</code>	Sort elements of an array	<code>np.sort(arr)</code>
<code>np.argsort()</code>	Return indices that would sort an array	<code>np.argsort(arr)</code>
<code>np.lexsort()</code>	Indirect sort using multiple keys	<code>np.lexsort((arr1, arr2))</code>
<code>np.sort_complex()</code>	Sort complex numbers by real then imaginary parts	<code>np.sort_complex(arr)</code>

✔ **Daily Use:** Ranking data, ordering results for display, preprocessing before binary search.

Searching & Indexing

Find positions or values in arrays based on criteria.

Function	Description	Example
<code>np.argmax()</code>	Index of the maximum value	<code>np.argmax(arr)</code>
<code>np.argmin()</code>	Index of the minimum value	<code>np.argmin(arr)</code>
<code>np.argwhere()</code>	Indices of elements matching condition	<code>np.argwhere(arr > 5)</code>
<code>np.nonzero()</code>	Indices of non-zero elements	<code>np.nonzero(arr)</code>
<code>np.where()</code>	Return indices where condition is true	<code>np.where(arr == 5)</code>

<code>np.searchsorted()</code>	Find index to insert element to maintain order	<code>np.searchsorted(arr, 7)</code>
<code>np.extract()</code>	Return elements that satisfy a condition	<code>np.extract(arr > 5, arr)</code>

✔ **Daily Use:** Data lookups, positional analysis, index-based slicing.

Unique & Set Operations

Perform mathematical set operations on arrays.

Function	Description	Example
<code>np.unique()</code>	Find unique elements	<code>np.unique(arr)</code>
<code>np.in1d()</code>	Check if elements of one array are in another	<code>np.in1d(arr1, arr2)</code>
<code>np.intersect1d()</code>	Intersection of two arrays	<code>np.intersect1d(arr1, arr2)</code>
<code>np.setdiff1d()</code>	Set difference (elements in arr1 not in arr2)	<code>np.setdiff1d(arr1, arr2)</code>
<code>np.setxor1d()</code>	Set symmetric difference	<code>np.setxor1d(arr1, arr2)</code>
<code>np.union1d()</code>	Union of two arrays	<code>np.union1d(arr1, arr2)</code>

✔ **Daily Use:** Removing duplicates, comparing datasets, merging data.

9. Random Module in NumPy

NumPy provides a powerful **random module** to generate random numbers, select random choices, and control randomness using seeds. These functions are widely used in **machine learning, simulations, probability, and data splitting**.

Random Number Generation

Generate random numbers from uniform, normal, and integer distributions.

Function	Description	Example
<code>np.random.rand()</code>	Generate random floats in range [0, 1)	<code>np.random.rand(3, 2)</code>
<code>np.random.randn()</code>	Generate samples from standard normal distribution	<code>np.random.randn(2, 3)</code>
<code>np.random.randint()</code>	Generate random integers in a given range	<code>np.random.randint(1, 100, (2, 2))</code>

Random Choice & Seed

Select random elements or fix randomness for reproducibility.

Function	Description	Example
<code>np.random.choice()</code>	Randomly select elements from an array	<code>np.random.choice([1,2,3], 5)</code>
<code>np.random.seed()</code>	Set seed for reproducibility of random results	<code>np.random.seed(42)</code>

✔ **Daily Use:** Train-test split, ML shuffling, simulations, reproducible experiments.

STAGE 3: Matrix & Linear Algebra (Level: Advanced)

Goal: Matrix operations used in ML & Stats

10. NumPy Matrix & Linear Algebra

NumPy provides powerful **matrix & linear algebra functions** for dot products, matrix multiplication, cross product, inner & outer products, and tensor operations. These functions are widely used in **machine learning, physics, computer graphics, and scientific computations**.

Dot Product & Matrix Multiplication

Perform dot products and matrix multiplications.

Function	Description	Example
<code>np.dot()</code>	Dot product of two arrays	<code>np.dot([1,2], [3,4])</code>
<code>np.matmul()</code>	Matrix product of two arrays	<code>np.matmul(A, B)</code>
<code>@ operator</code>	Shorthand for matrix multiplication	<code>A @ B</code>

✖ Element-wise Multiplication

Multiply elements of arrays position-wise.

Function	Description	Example
<code>np.multiply()</code>	Element-wise multiplication	<code>np.multiply([1,2], [3,4])</code>
<code>*</code>	Direct element-wise multiply	<code>A * B</code>

Inner & Outer Products

Compute inner and outer products between vectors.

Function	Description	Example
<code>np.inner()</code>	Inner product of arrays	<code>np.inner([1,2], [3,4])</code>
<code>np.outer()</code>	Outer product of arrays	<code>np.outer([1,2], [3,4])</code>

✓ **Daily Use:** Machine Learning (linear models, PCA, Neural Nets), Physics simulations, 3D graphics, and Scientific Computing.

11. Broadcasting in NumPy

☆☆ Broadcasting Rules (Golden Rules)

Rule 1: Compare from Right

Rightmost dimension se shapes compare karo. Agar left side me missing dimension ho → 1 maan lo.

```
import numpy as np
a = np.array([1,2,3])      # shape (3,)
b = np.array([[10],[20]])  # shape (2,1)
```

Consider shapes:
a → (1,3)
b → (2,1)

Rule 2: Same Dimension → OK

Agar dono shapes ke axis same hai → broadcasting possible.

```
a = np.array([1,2,3])
b = np.array([10,20,30])
print(a + b)
```

Shapes: (3,) vs (3,)
Output: [11, 22, 33] ✓

Rule 3: Dimension 1 → Stretch

1 ko repeat karke dusre array ke size tak stretch kar diya jata hai.

```
a = np.array([[1],[2],[3]]) # shape (3,1)
b = np.array([10,20,30])    # shape (3,)
print(a + b)
```

Shapes:
a: (3,1)
b: (1,3)

Output:
[[11 21 31]
[12 22 32]
[13 23 33]]

Rule 4: Mismatch & No 1 → ERROR

Agar dimensions match nahi karte aur 1 bhi nahi hai → broadcasting fail.

```
a = np.array([1,2,3]) # shape (3,)
b = np.array([1,2])   # shape (2,)
print(a + b)
```

Shapes: (3,) vs (2,)
Output: ValueError: operands could not be broadcast together

🔑 Short Recap

Step 1: Rightmost dimension se compare

Step 2: Same → ✓

Step 3: Ek dimension 1 → Stretch → ✓

Step 4: Na same, na 1 → ✗ Error

11. Working with Missing Data

Detect, represent, and replace missing values in arrays.

Function	Description	Example
<code>np.isnan()</code>	Check if elements are NaN	<code>np.isnan([1, np.nan, 3])</code>
<code>np.nan</code>	Represents a missing value	<code>x = np.array([1, np.nan, 3])</code>
<code>np.nan_to_num()</code>	Replace NaN with a specified number	<code>np.nan_to_num([1, np.nan, 3], nan=0)</code>

12. Conversion & Type Casting

Convert array types, check data types, and convert arrays to lists.

Function / Method	Description	Example
<code>.astype()</code>	Convert array to specified data type	<code>arr.astype('float64')</code>
<code>.tolist()</code>	Convert array to Python list	<code>arr.tolist()</code>
<code>.dtype</code>	Get the data type of array elements	<code>arr.dtype</code>

Data Type	Description	Example
<code>int / int32 / int64</code>	Signed integer	<code>arr.astype(int)</code>
<code>float / float32 / float64</code>	Floating point	<code>arr.astype(float)</code>
<code>complex / complex64 / complex128</code>	Complex numbers	<code>arr.astype(complex)</code>
<code>bool</code>	Boolean (True/False)	<code>arr.astype(bool)</code>
<code>str / string_</code>	String	<code>arr.astype(str)</code>
<code>object</code>	Generic Python object	<code>arr.astype(object)</code>
<code>uint / uint8/16/32/64</code>	Unsigned integer	<code>arr.astype(np.uint8)</code>
<code>datetime64</code>	Date-time	<code>arr.astype('datetime64[D]')</code>
<code>timedelta64</code>	Time difference	<code>arr.astype('timedelta64[D]')</code>

13. Some Important Topics in NumPy

Frequently used NumPy functions for data operations, statistics, and transformations.

Function / Method	Description	Example
<code>np.append()</code>	Add values to the end of an array	<code>np.append(arr, [7,8])</code>
<code>np.concatenate()</code>	Join two or more arrays along an axis	<code>np.concatenate([arr1, arr2])</code>
<code>np.cumsum()</code> , <code>np.cumprod()</code>	Cumulative sum / cumulative product of elements	<code>np.cumsum(arr)</code>
<code>np.percentile()</code>	Compute nth percentile of data	<code>np.percentile(arr, 50)</code>
<code>np.histogram()</code>	Compute histogram of data	<code>np.histogram(arr, bins=5)</code>
<code>np.corrcoef()</code>	Correlation coefficient matrix	<code>np.corrcoef(arr1, arr2)</code>
<code>np.flip()</code>	Reverse array elements along axis	<code>np.flip(arr)</code>
<code>np.put()</code>	Replace values at given indices	<code>np.put(arr, [0,2], [99,100])</code>
<code>np.delete()</code>	Delete elements along axis	<code>np.delete(arr, 1)</code>
<code>np.clip()</code>	Limit values in array within range	<code>np.clip(arr, 0, 10)</code>

Thank You!

This notebook was created with ♥ and curiosity to explore the power of **NumPy** and Data Science.

"Learning never exhausts the mind, it only fuels the journey ahead."

Keep Learning • Keep Exploring • Keep Growing